LAPORAN PRAKTIKUM

POSTTEST 7

ALGORITMA PEMROGRAMAN DASAR



Disusun oleh:

Muhammad Rifqi Ramadhan (2609106040)

Kelas (B1‘26)

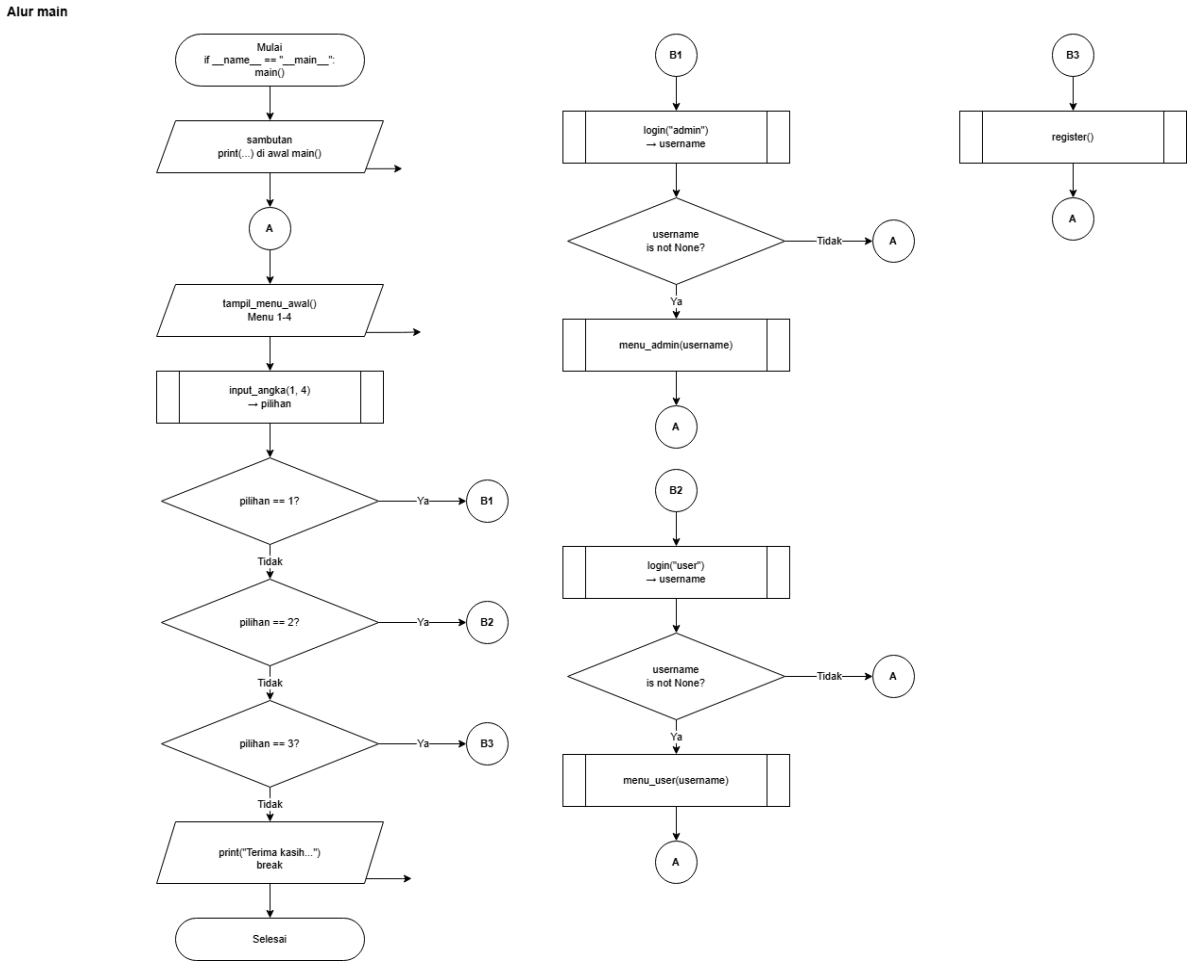
PROGRAM STUDI INFORMATIKA

UNIVERSITAS MULAWARMAN

SAMARINDA

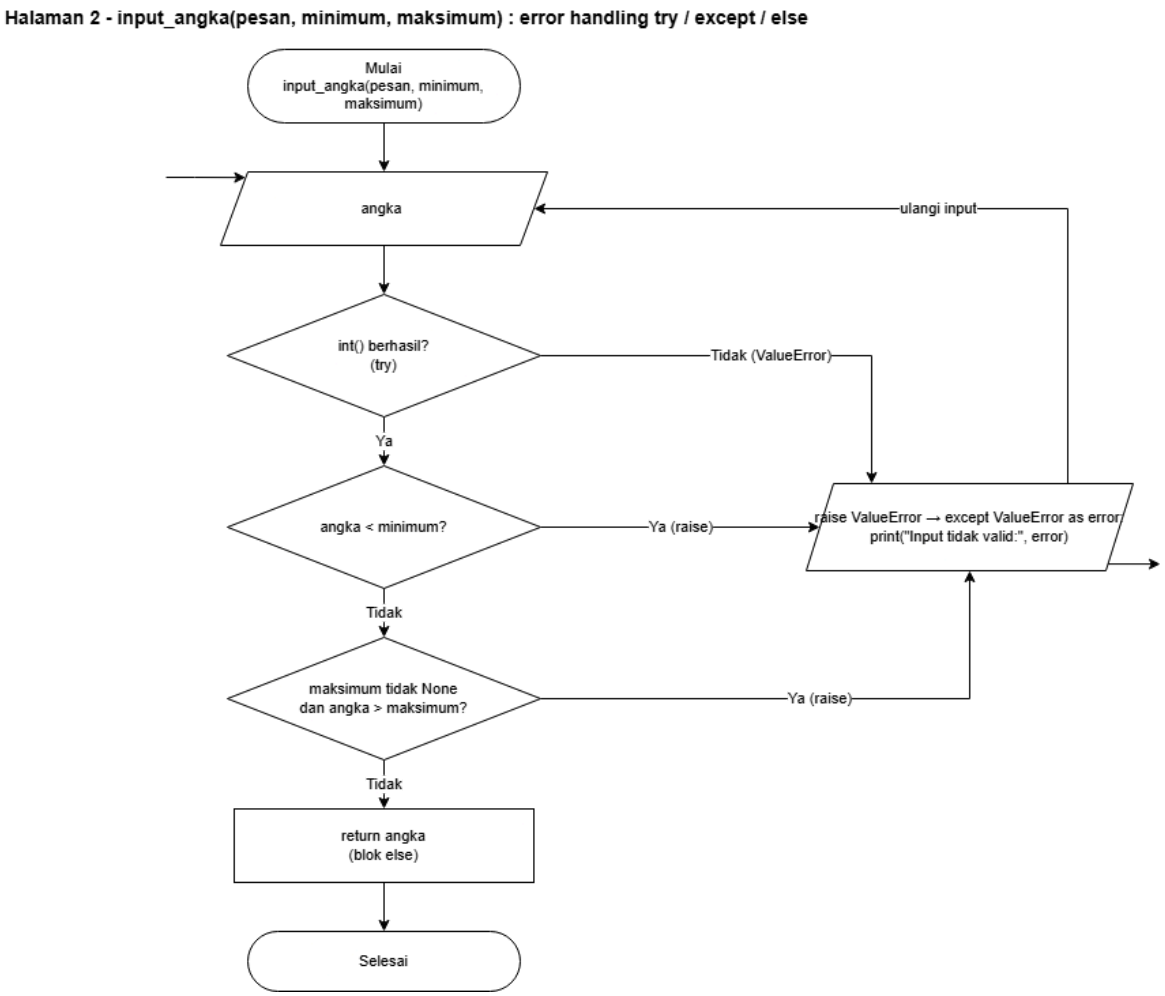
2026

1. Flowchart



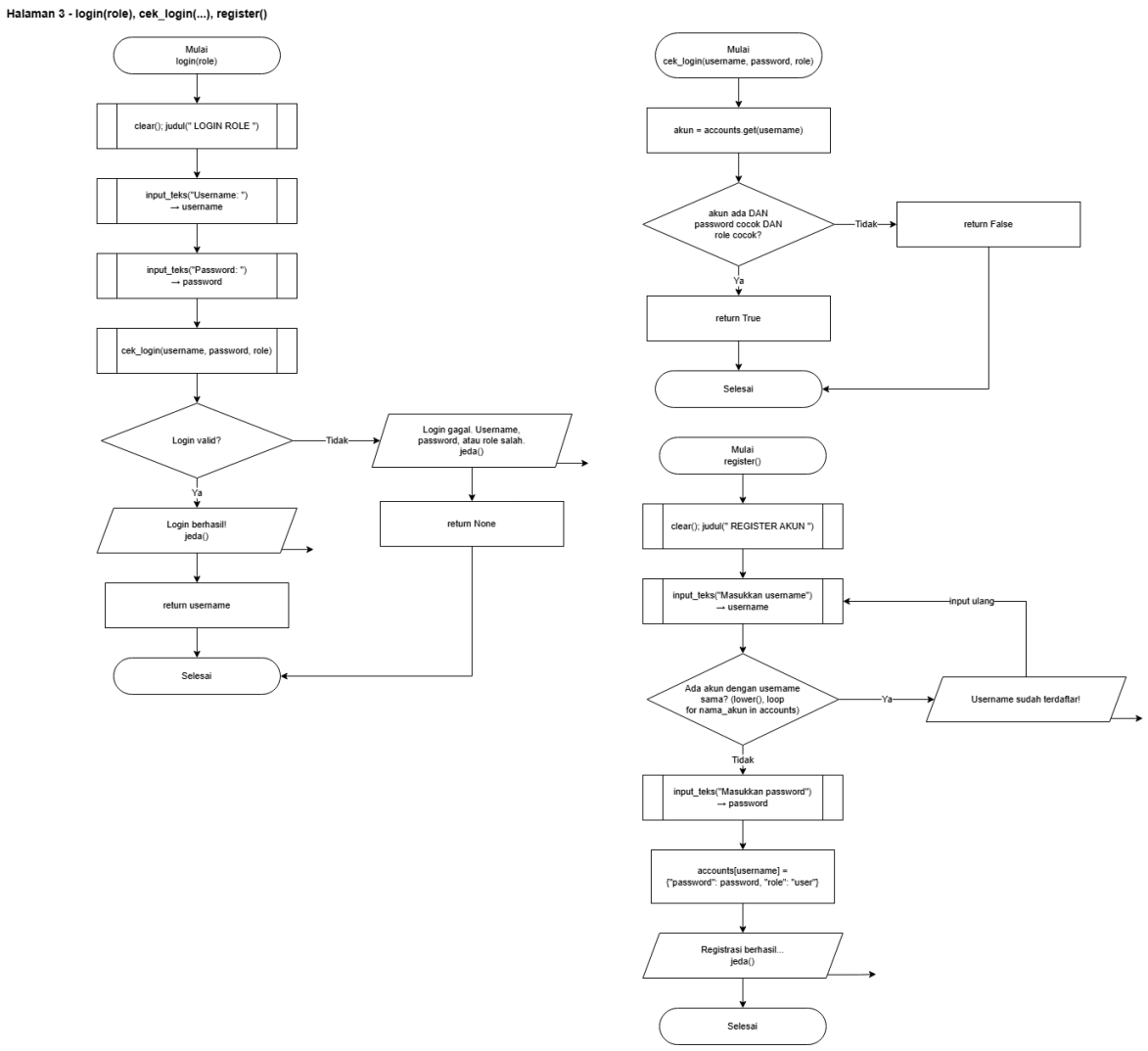
Gambar 1.1 Flowchart Program VlaoTracker - Alur Utama Program (Main) Draw.io

Program dimulai dengan menampilkan pesan sambutan, lalu masuk ke loop menu awal yang menampilkan empat pilihan. Pilihan angka divalidasi lewat input_angka. Pilihan 1 menjalankan login admin dan membuka menu_admin bila berhasil. Pilihan 2 menjalankan login user dan membuka menu_user bila berhasil. Pilihan 3 memanggil register. Setelah tiap cabang selesai, alur kembali ke menu awal. Pilihan 4 mencetak ucapan terima kasih dan mengakhiri program.



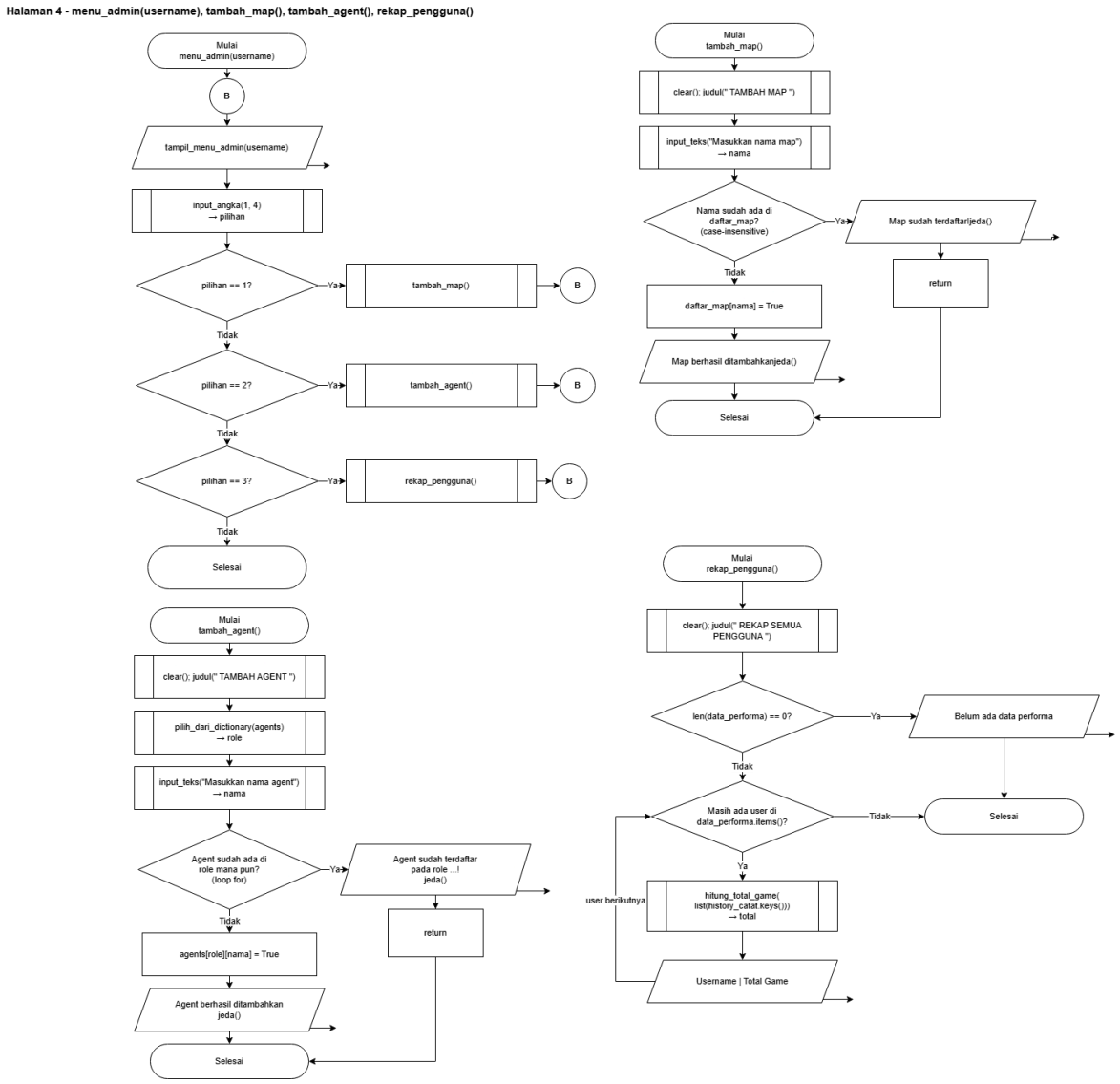
Gambar 1.2 Flowchart Program VlaoTracker - Fungsi input_angka (Error Handling) Draw.io

Fungsi ini meminta input, mengubahnya menjadi integer, lalu memeriksa batas minimum dan maksimum. Jika konversi gagal atau angka di luar batas, program memunculkan ValueError, menampilkan pesan “Input tidak valid”, dan meminta input lagi. Jika semua pemeriksaan lolos, angka dikembalikan lewat blok else. Pola yang sama dipakai oleh input_teks dan konfirmasi.



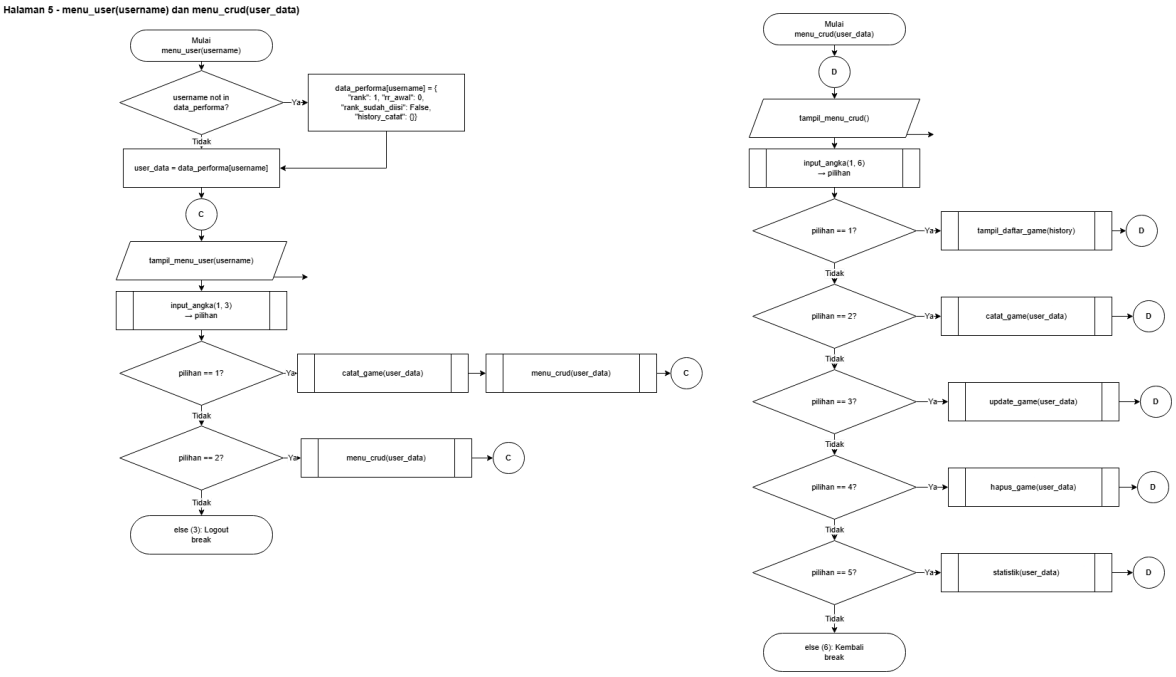
Gambar 1.3 Flowchart Program VlaoTracker - Login, Cek Login, dan Register Draw.io

login(role) menampilkan judul, meminta username dan password, lalu memanggil cek_login. Jika valid, fungsi mencetak pesan berhasil dan mengembalikan username. Jika tidak, fungsi mencetak pesan gagal dan mengembalikan None. cek_login mengambil akun dari dictionary accounts dan mengembalikan True hanya bila akun ada dan password serta role cocok. register() meminta username dan mengulang selama username sudah terdaftar (tanpa membedakan huruf besar-kecil). Setelah itu fungsi meminta password dan menyimpan akun baru dengan role “user”.



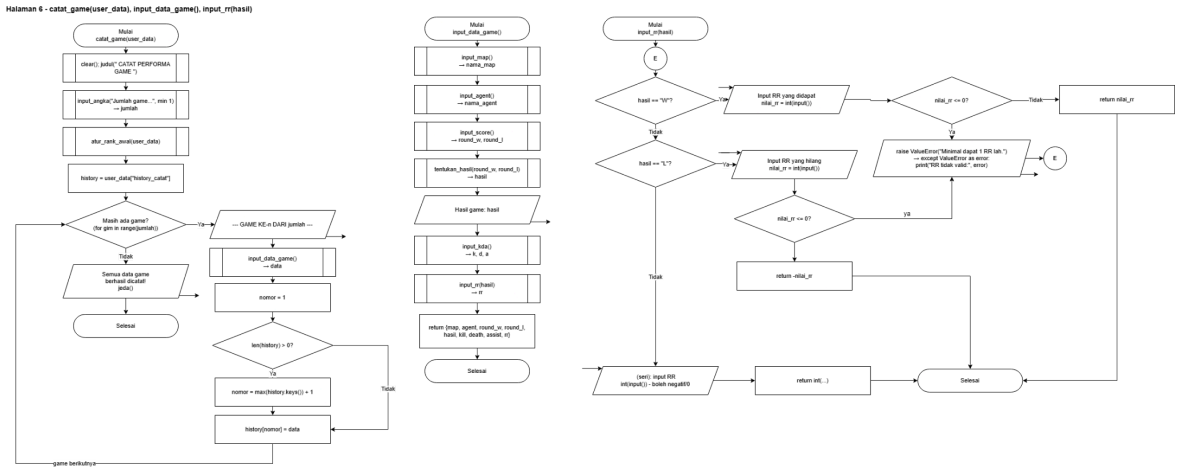
Gambar 1.4 Flowchart Program VlaoTracker - Menu Admin Draw.io

menu_admin menampilkan menu dalam loop dan memanggil fitur sesuai pilihan, yaitu tambah_map, tambah_agent, atau rekap_pengguna. Pilihan keempat keluar dari loop sebagai logout. tambah_map dan tambah_agent sama-sama menolak nama yang sudah ada (case-insensitive) sebelum menyimpannya. Untuk agent, admin memilih role terlebih dahulu. rekap_pengguna menampilkan semua user di data_performa beserta total game masing-masing, dan memakai fungsi rekursif hitung_total_game untuk menghitungnya.



Gambar 1.5 Flowchart Program VlaoTracker - Dashboard User dan Menu CRUD Draw.io

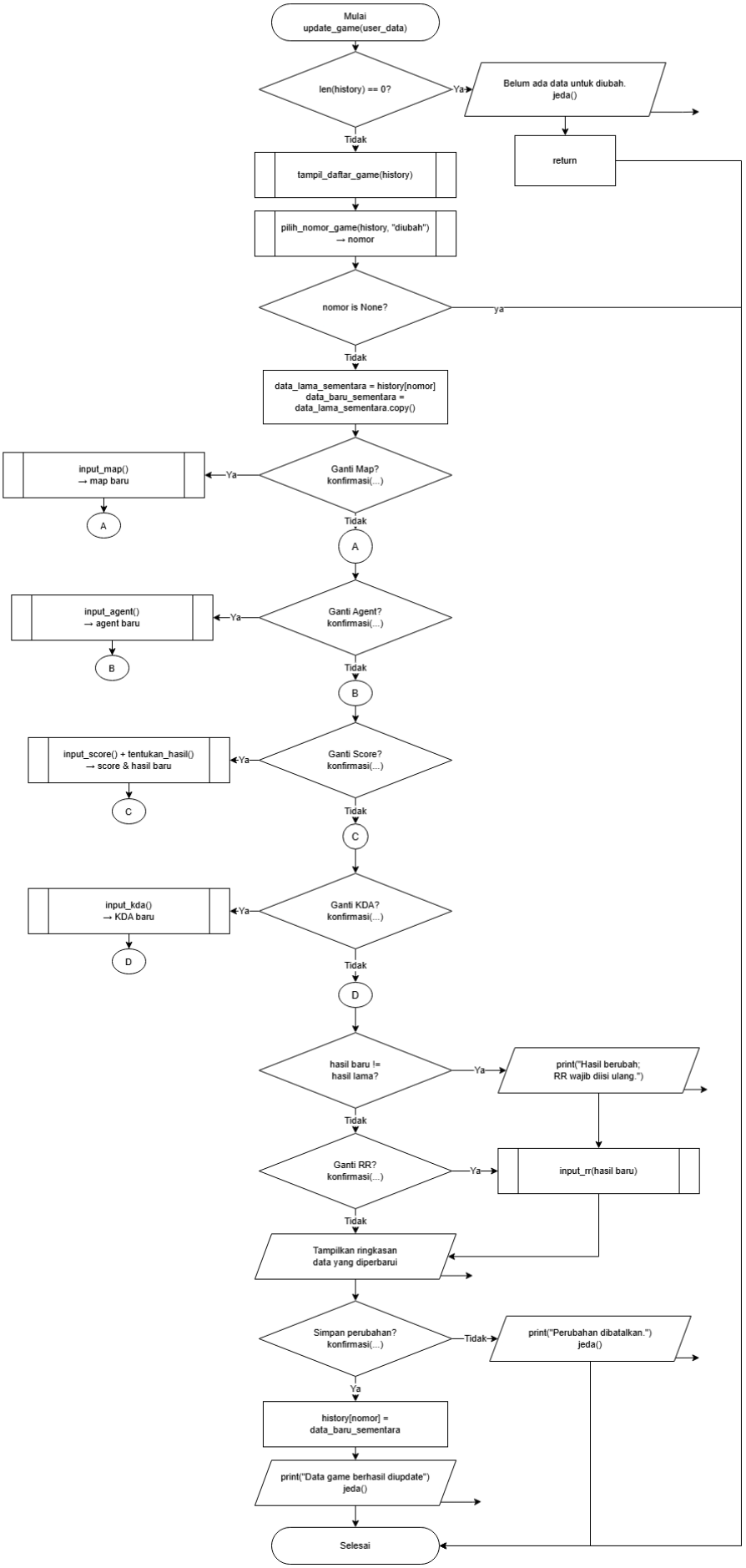
menu_user membuat struktur data awal untuk user baru bila belum ada di data_performa, lalu menampilkan menu dalam loop. Pilihan 1 mencatat game lebih dulu lalu langsung membuka menu_crud. Pilihan 2 langsung membuka menu_crud. Pilihan 3 adalah logout. menu_crud berjalan dalam loop dengan enam pilihan: lihat, tambah, update, hapus, statistik, dan kembali ke dashboard. Tiap pilihan memanggil fungsi yang sesuai.

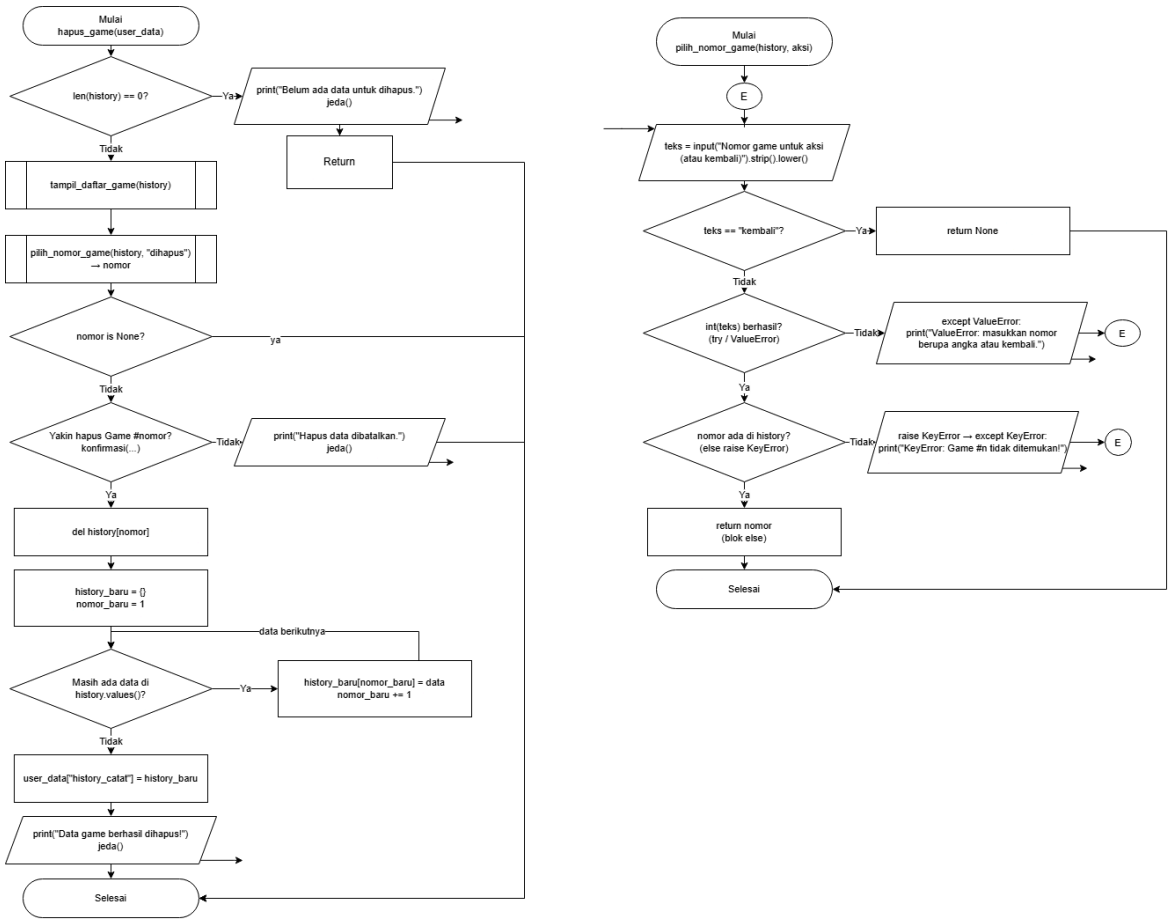


Gambar 1.6 Flowchart Program VlaoTracker - Catat Performa Game Draw.io

catat_game meminta jumlah game yang akan dicatat, lalu memanggil atur_rank_awal. Untuk setiap game, fungsi memanggil input_data_game, menentukan nomor game berikutnya, dan menyimpan data ke history. Nomor dimulai dari 1 atau dari nomor terbesar ditambah satu supaya tidak menimpa data lama. input_data_game mengumpulkan map, agent, skor, hasil, KDA, dan RR, lalu mengembalikannya sebagai dictionary. input_rr memvalidasi RR sesuai hasil. Untuk menang atau kalah RR wajib lebih dari nol, dan RR kalah disimpan negatif. Untuk seri RR boleh berapa saja. Input yang salah memicu ValueError dan diulang.

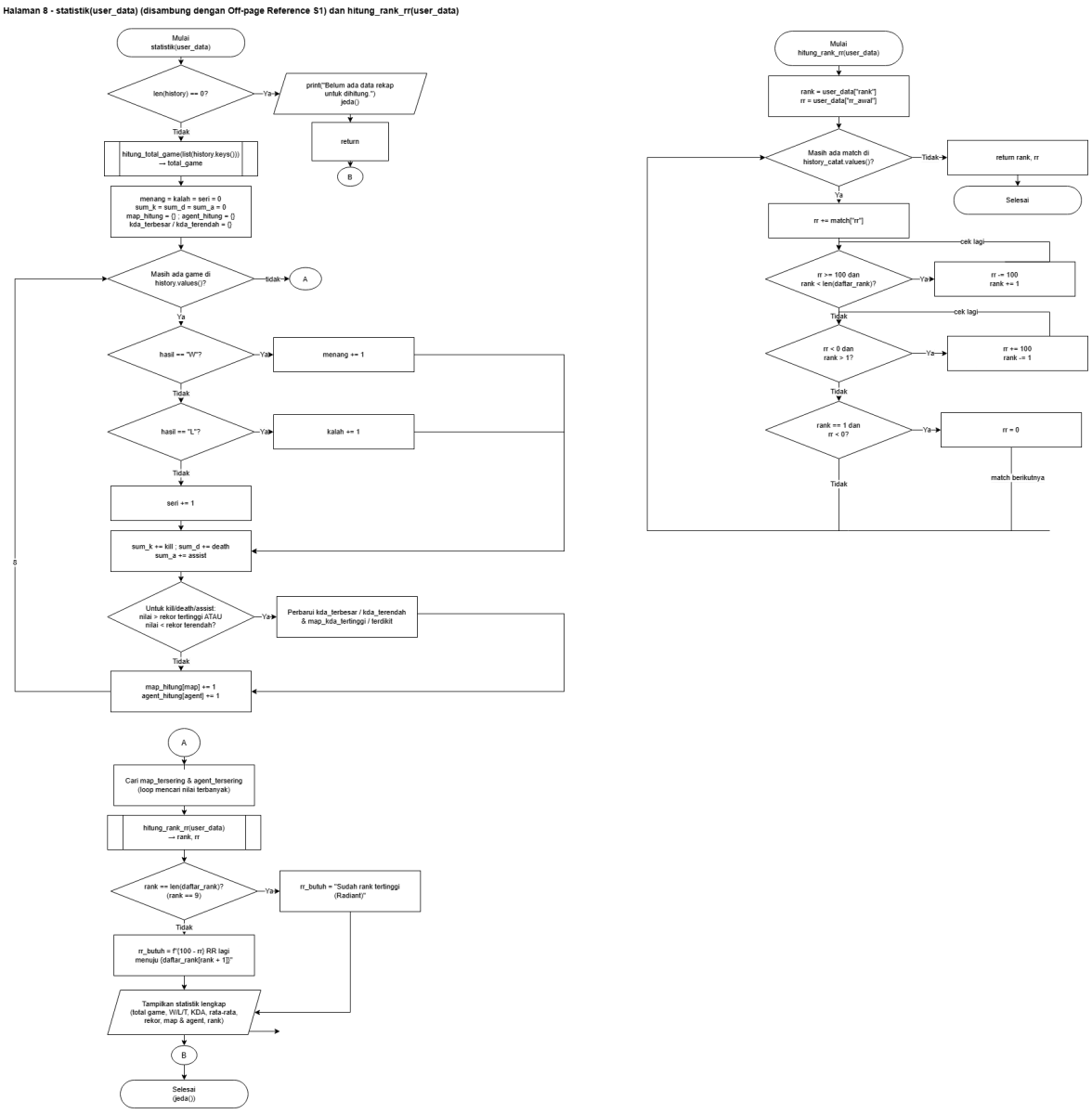
Halaman 7 - update_game(user_data), hapus_game(user_data), pilih_nomor_game(history, aksi)





Gambar 1.6 Flowchart Program VlaoTracker - Catat Performa Game Draw.io

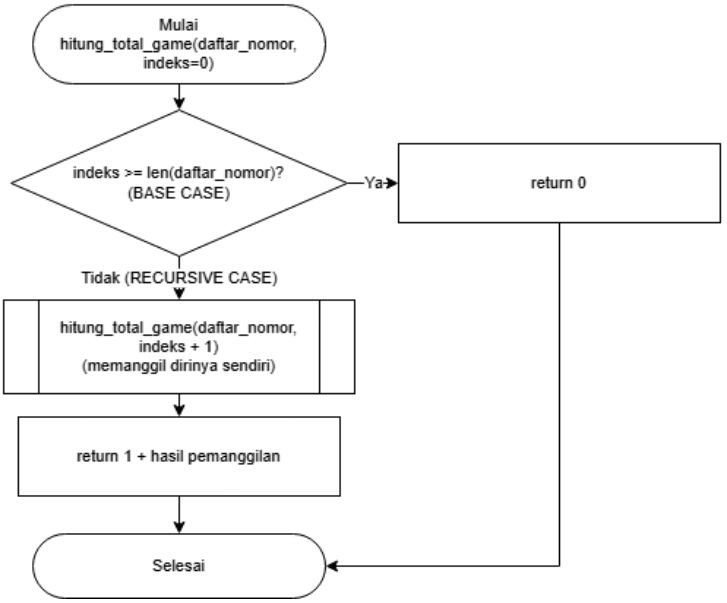
update_game menampilkan daftar game, memilih nomor lewat pilih_nomor_game, lalu menyalin data lama ke dictionary sementara. User ditanya satu per satu apakah ingin mengganti map, agent, skor, atau KDA. Jika hasil menang/kalah/seri berubah, RR wajib diisi ulang. Jika tidak berubah, RR hanya diganti bila user setuju. Setelah ringkasan ditampilkan, data asli baru diganti bila user mengonfirmasi penyimpanan. hapus_game meminta konfirmasi, menghapus game terpilih, lalu menomori ulang seluruh game dari 1. pilih_nomor_game menerima angka atau kata “kembali”. Ia menangani ValueError untuk input non-angka dan KeyError untuk nomor yang tidak ada, lalu mengulang input.



Gambar 1.8 Flowchart Program VlaoTracker - Statistik dan Perhitungan Rank Draw.io

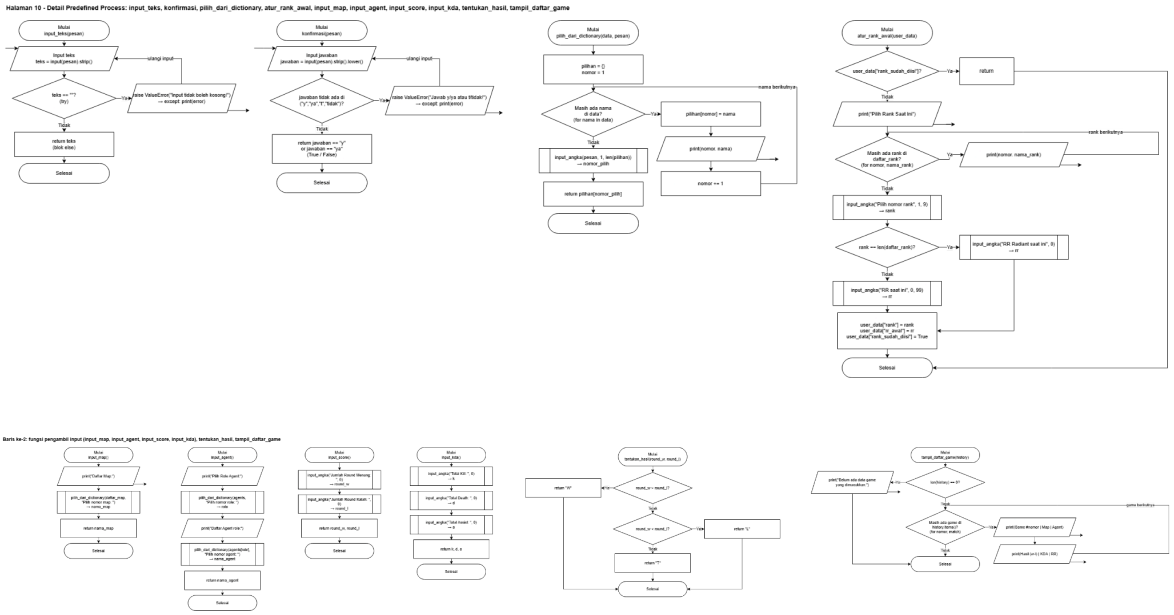
statistik memeriksa apakah ada data. Jika ada, fungsi menghitung total game dengan fungsi rekursif, lalu melakukan loop pada seluruh game. Dalam loop itu fungsi menghitung menang, kalah, dan seri, menjumlahkan KDA, memperbarui rekor KDA tertinggi dan terendah, serta menghitung frekuensi map dan agent. Setelah loop, alur berlanjut lewat Off-page Reference S1. Di halaman lanjutan, fungsi mencari map dan agent yang paling sering dimainkan, memanggil hitung_rank_rr untuk rank dan RR terkini, lalu menghitung sisa RR menuju rank berikutnya dan menampilkan seluruh statistik. hitung_rank_rr menjumlahkan RR tiap game dari rank awal. Rank naik bila RR mencapai 100 atau lebih dan turun bila RR negatif, dengan batas rank 1 dan rank tertinggi.

Halaman 9 - hitung_total_game(daftar_nomor, indeks=0) : FUNGSI REKURSIF



Gambar 1.9 Flowchart Program VlaoTracker - Fungsi Rekursif hitung_total_game Draw.io

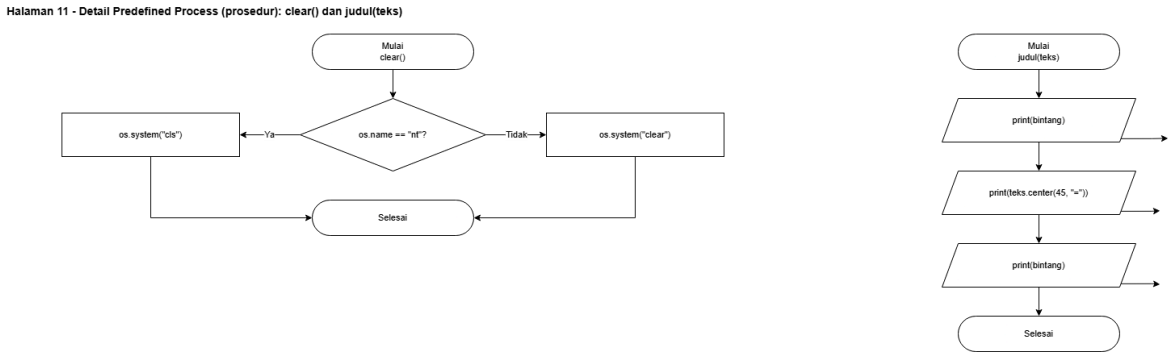
hitung_total_game menghitung jumlah game dengan memanggil dirinya sendiri. Base case-nya adalah ketika indeks sudah mencapai panjang daftar, yang mengembalikan 0. Selama belum tercapai, fungsi mengembalikan 1 ditambah hasil pemanggilan dengan indeks berikutnya. Untuk dua game hasilnya 1 + (1 + 0) = 2.



Gambar 1.10 Flowchart Program VlaoTracker - Fungsi Input (Predefined Process) Draw.io

Halaman ini merinci fungsi-fungsi yang dipanggil sebagai Predefined Process. input_teks dan konfirmasi mengulang input sampai teks tidak kosong atau jawabannya salah satu dari y, ya, t, tidak.

pilih_dari_dictionary mencetak isi dictionary bernomor, meminta nomor, lalu mengembalikan nama yang dipilih. atur_rank_awal hanya berjalan sekali, yaitu saat rank belum pernah diisi. Fungsi ini meminta rank dan RR awal, dengan rentang RR yang berbeda untuk rank tertinggi. input_map, input_agent, input_score, dan input_kda adalah pembungkus untuk pilih_dari_dictionary dan input_angka. tentukan_hasil membandingkan skor dan mengembalikan “W”, “L”, atau “T”. tampil_daftar_game mencetak seluruh game atau pesan bahwa data masih kosong.



Gambar 1.11 Flowchart Program VlaoTracker - Prosedur Bantu (clear dan judul) Draw.io

Halaman ini merinci fungsi-fungsi yang dipanggil sebagai Predefined Process. input_teks dan konfirmasi mengulang input sampai teks tidak kosong atau jawabannya salah satu dari y, ya, t, tidak. pilih_dari_dictionary mencetak isi dictionary bernomor, meminta nomor, lalu mengembalikan nama yang dipilih. atur_rank_awal hanya berjalan sekali, yaitu saat rank belum pernah diisi. Fungsi ini meminta rank dan RR awal, dengan rentang RR yang berbeda untuk rank tertinggi. input_map, input_agent, input_score, dan input_kda adalah pembungkus untuk pilih_dari_dictionary dan input_angka. tentukan_hasil membandingkan skor dan mengembalikan “W”, “L”, atau “T”. tampil_daftar_game mencetak seluruh game atau pesan bahwa data masih kosong.

VlaoTracker adalah program berbasis terminal untuk mencatat dan menganalisis performa bermain Valorant berdasarkan map, agent (dikelompokkan per role), skor round, KDA, dan perubahan RR. Program lalu menghitung statistik serta estimasi rank pengguna. Program mendukung dua peran, yaitu admin dan pengguna biasa. Setelah menampilkan pesan sambutan, program menampilkan menu awal berisi login admin, login member, register, dan keluar. Login memeriksa kecocokan username, password, dan role sebelum membuka fitur sesuai perannya, sedangkan register hanya membuat akun dengan role pengguna biasa. Admin dapat menambah map dan agent baru, dengan pengecekan nama ganda tanpa membedakan huruf besar-kecil, serta melihat rekap jumlah game setiap pengguna. Pengguna mengisi rank dan RR awal satu kali, lalu mencatat game dengan memilih map, role, dan agent, kemudian memasukkan skor round, KDA, dan RR. Hasil game (menang, kalah, atau seri) ditentukan otomatis dari

skor round, dan input RR menyesuaikan hasil tersebut. RR menang dan kalah wajib lebih dari nol, dengan RR kalah disimpan sebagai nilai negatif, sedangkan RR seri boleh bernilai berapa pun. Seluruh input divalidasi dengan error handling (try-except-else dan raise) yang menangani ValueError dan KeyError, sehingga input kosong atau salah diminta ulang. Data game dapat dilihat, ditambah, diubah, dan dihapus (CRUD). Perubahan data diproses di dictionary sementara dan baru disimpan setelah dikonfirmasi, sedangkan penghapusan memakai konfirmasi dan menomori ulang game. Riwayat performa disimpan dalam memori menggunakan dictionary bersarang (nested dictionary), sehingga data hilang ketika program ditutup. Program menampilkan rekap statistik berupa total game, jumlah menang/kalah/seri, total dan rata-rata KDA, rekor kill, death, dan assist terbanyak maupun terdikit beserta map dan agent-nya, map dan agent paling sering dimainkan, serta rank saat ini yang dihitung dari akumulasi RR, termasuk promosi dan demosi rank dan sisa RR menuju rank berikutnya. Jumlah game dihitung dengan fungsi rekursif.

2. Deskripsi Singkat Program

Tujuan VlaoTracker dibuat untuk membantu pemain Valorant mencatat dan memantau performa permainan ranked.

Manfaat utama, pencatatan performa game, dapat menambah, melihat, mengubah, dan menghapus data game yang sudah tercatat, sehingga kesalahan input bisa diperbaiki, rekap otomatis disini pemain bisa mengevaluasi permainannya. Dan untuk admin bisa melihat berapa banyak yang memakai program ini.

3. Source Code

A. Fitur Tambah Data

Fitur ini dipakai pengguna untuk mencatat satu atau beberapa game sekaligus. Program meminta jumlah game, mengatur rank awal (hanya sekali), lalu memanggil input_data_game() untuk setiap game. Nomor game dibuat dari nomor terbesar ditambah satu supaya tidak menimpa data lama.

```
def catat_game(user_data):
    clear()
    judul(" CATAT PERFORMA GAME ")
    jumlah = input_angka("Jumlah game yang ingin dicatat: ", 1)
    atur_rank_awal(user_data)
    history = user_data["history_catat"]

    for gim in range(jumlah):
        print(f"\n--- GAME KE-{gim + 1} DARI {jumlah} ---")
        data = input_data_game()
        nomor_data_baru_sementara = 1

        if len(history) > 0:
            nomor_data_baru_sementara = max(history.keys()) + 1
        history[nomor_data_baru_sementara] = data
    print("\nSemua data game berhasil dicatat!")
    jeda()
```

```
def input_data_game():
    # lokal variabel
    nama_map = input_map()
```

```
    # lokal variabel
    nama_agent = input_agent()
    # lokal variabel
    round_w, round_l = input_score()
    # lokal variabel
    hasil = tentukan_hasil(round_w, round_l)
    print(f"Hasil game: {hasil}")
    k, d, a = input_kda()
    rr = input_rr(hasil)
    return {"map": nama_map, "agent": nama_agent, "round_w": round_w,
"round_l": round_l, "hasil": hasil, "kill": k, "death": d, "assist": a,
"rr": rr}
```

B. Lihat Data

Fitur ini menampilkan semua game yang sudah tercatat milik pengguna. Jika belum ada data, program menampilkan pesan bahwa data masih kosong.

```
def tampil_daftar_game(history):
    if len(history) == 0:
        print("Belum ada data game yang dimasukkan.")
    else:
        for nomor, match in history.items():
            print(f"Game #{nomor} | Map: {match['map']} |
Agent: {match['agent']}")
            print(f"  Hasil: {match['hasil']}
({match['round_w']}-{match['round_l']}) "
                  f"| KDA:
{match['kill']}/{match['death']}/{match['assist']} "
                  f"| RR: {match['rr']:+}")
```

C. Update Data

Fitur ini mengubah data game yang sudah dicatat. Perubahan dikerjakan pada dictionary sementara (hasil copy()), sehingga data asli baru berubah setelah pengguna mengonfirmasi penyimpanan. Jika hasil game berubah (misalnya dari menang menjadi kalah), RR wajib diisi ulang.

```
    def update_game(user_data):
     clear()
```

```
    judul(" UPDATE DATA GAME ")
    history = user_data["history_catat"]

    if len(history) == 0:
        print("Belum ada data untuk diubah.")
        jeda()
        return

    tampil_daftar_game(history)
    nomor = pilih_nomor_game(history, "diubah")

    if nomor is None:
        return

    data_lama_sementara = history[nomor]

    data_baru_sementara = data_lama_sementara.copy()

    print(f"\nMengubah Game #{nomor}")
    if konfirmasi(f"Ganti Map ({data_lama_sementara['map']})?
(y/t): "):
        data_baru_sementara["map"] = input_map()

    if konfirmasi(f"Ganti Agent
({data_lama_sementara['agent']})? (y/t): "):
        data_baru_sementara["agent"] = input_agent()

    if konfirmasi(f"Ganti Score
({data_lama_sementara['round_w']}-{data_lama_sementara['round
_l']})? (y/t): "):
        data_baru_sementara["round_w"],
data_baru_sementara["round_l"] = input_score()
        data_baru_sementara["hasil"] =
tentukan_hasil(data_baru_sementara["round_w"],
data_baru_sementara["round_l"])

    if konfirmasi(f"Ganti KDA
({data_lama_sementara['kill']}/{data_lama_sementara['death']}
/{data_lama_sementara['assist']})? (y/t): "):
        data_baru_sementara["kill"],
data_baru_sementara["death"], data_baru_sementara["assist"] =
input_kda()
```

```
    if data_baru_sementara["hasil"] !=
data_lama_sementara["hasil"]:
        print(f"Hasil berubah {data_lama_sementara['hasil']}
menjadi {data_baru_sementara['hasil']}; RR wajib diisi
ulang.")
        data_baru_sementara["rr"] =
input_rr(data_baru_sementara["hasil"])

    elif konfirmasi(f"Ganti RR
({data_lama_sementara['rr']:+})? (y/t): "):
        data_baru_sementara["rr"] =
input_rr(data_baru_sementara["hasil"])

    print("\n--- RINGKASAN DATA YANG DIPERBARUI ---")
    print(f"Map: {data_baru_sementara['map']} | Agent:
{data_baru_sementara['agent']}")
    print(f"Score:
{data_baru_sementara['round_w']}-{data_baru_sementara['round_
l']} ({data_baru_sementara['hasil']})")
    print(f"KDA:
{data_baru_sementara['kill']}/{data_baru_sementara['death']}/
{data_baru_sementara['assist']} | RR:
{data_baru_sementara['rr']:+}")

    if konfirmasi("Simpan perubahan? (y/t): "):
        history[nomor] = data_baru_sementara
        print("Data game berhasil diupdate")
    else:
        print("Perubahan dibatalkan.")
    jeda()
```

D. Hapus Data

Fitur ini menghapus satu game setelah pengguna mengonfirmasi. Setelah data dihapus, nomor game disusun ulang dari 1 supaya urutannya tidak berlubang.

```
def hapus_game(user_data):
    clear()
    judul(" HAPUS DATA GAME ")
    history = user_data["history_catat"]
    if len(history) == 0:
        print("Belum ada data untuk dihapus.")
        jeda()
```

```
        return

    tampil_daftar_game(history)
    nomor = pilih_nomor_game(history, "dihapus")

    if nomor is None:
        return
    if konfirmasi(f"Yakin hapus Game #{nomor}? (y/t): "):
        del history[nomor]
        history_data_baru_sementara = {}
        nomor_data_baru_sementara = 1

        for data in history.values():
            history_data_baru_sementara[nomor_data_baru_sementara] =
data
            nomor_data_baru_sementara += 1
        user_data["history_catat"] = history_data_baru_sementara
        print("Data game berhasil dihapus!")
    else:
        print("Hapus data dibatalkan.")
    jeda()
```

E. Fitur Statistik dan Perhitungan Rank

Fitur ini merekap performa pengguna dalam satu tampilan. Isinya jumlah menang, kalah, dan seri, total dan rata-rata KDA, rekor kill, death, dan assist tertinggi maupun terendah, serta map dan agent yang paling sering dimainkan. Rank saat ini dihitung oleh hitung_rank_rr() dari akumulasi RR. Rank naik jika RR mencapai 100 atau lebih, dan turun jika RR negatif.

```
    for match in history.values():
        if match["hasil"] == "W":
            menang += 1
        elif match["hasil"] == "L":
            kalah += 1
        else:
            seri += 1

        sum_k += match["kill"]
        sum_d += match["death"]
        sum_a += match["assist"]
        teks_map = f"Map {match['map']} (Agent
{match['agent']})"

        for key_kda in ("kill", "death", "assist"):
```

```
            if pertama or match[key_kda] >
kda_terbesar[key_kda]:
                kda_terbesar[key_kda] = match[key_kda]
                map_kda_tertinggi[key_kda] = teks_map
            if pertama or match[key_kda] <
kda_terendah[key_kda]:
                kda_terendah[key_kda] = match[key_kda]
                map_kda_terdikit[key_kda] = teks_map
        pertama = False
        map_hitung[match["map"]] =
map_hitung.get(match["map"], 0) + 1
        agent_hitung[match["agent"]] =
agent_hitung.get(match["agent"], 0) + 1

def hitung_rank_rr(user_data):
    rank = user_data["rank"]
    rr = user_data["rr_awal"]
    for match in user_data["history_catat"].values():
        rr += match["rr"]
        while rr >= 100 and rank < len(daftar_rank):
            rr -= 100
            rank += 1
        while rr < 0 and rank > 1:
            rr += 100
            rank -= 1
        if rank == 1 and rr < 0:
            rr = 0
    return rank, rr
```

F. Fitur Login, Register, dan Multiuser

Fitur ini membedakan hak akses admin dan pengguna biasa. cek_login() mencocokkan username, password, dan role di dictionary accounts. register() menolak username yang sudah ada tanpa membedakan huruf besar-kecil, lalu menyimpan akun baru dengan role “user”.

```
  def cek_login(username, password, role):
    akun = accounts.get(username)
    return akun is not None and akun["password"] == password and
akun["role"] == role

def register():
    clear()
```

```
    judul(" REGISTER AKUN BARU ")
    while True:
        # lokal variabel 1
        username = input_teks("Masukkan username baru: ")
        # lokal variabel 2
        sudah_ada = False
        for nama_akun in accounts:
            if nama_akun.lower() == username.lower():
                sudah_ada = True
        if sudah_ada:
            print("Username sudah terdaftar!")
        else:
            break
    password = input_teks("Masukkan password baru: ")
    accounts[username] = {"password": password, "role": "user"}
    print("Registrasi berhasil, silakan login.")
    jeda()

def login(role):
    clear()
    judul(f" LOGIN {role.upper()} ")
    # lokal variabel 3
    username = input_teks("Username: ")
    # lokal variabel 4
    password = input_teks("Password: ")
    if cek_login(username, password, role):
        print("Selamat login berhasil")
        jeda()
        return username
    else:
        print("Login gagal, mungkin ada yang salah 🤔")
        jeda()
        return None
```

G. Fitur Admin (Tambah Map dan Rekap Pengguna)

Admin dapat menambah map baru dengan pengecekan nama ganda tanpa membedakan huruf besar-kecil. Fitur tambah agent bekerja dengan cara yang sama, ditambah pemilihan role. Admin juga dapat melihat rekap jumlah game setiap pengguna, yang dihitung dengan fungsi rekursif.

```
def tambah_map():
    clear()
    judul(" TAMBAH MAP ")
    # lokal variabel 5
    nama_data_baru_sementara = input_teks("Masukkan nama map
data_baru_sementara: ")
```

```
    for nama_map in daftar_map:
        if nama_map.lower() == nama_data_baru_sementara.lower():
            print("Map sudah terdaftar!")
            jeda()
            return
    daftar_map[nama_data_baru_sementara] = True
    print(f"Map '{nama_data_baru_sementara}' berhasil ditambahkan.")
    jeda()

def tambah_agent():
    clear()
    judul(" TAMBAH AGENT ")
    role = pilih_dari_dictionary(agents, "Pilih nomor role: ")
    nama_data_baru_sementara = input_teks("Masukkan nama agent
data_baru_sementara: ")
    for nama_role, daftar_agent in agents.items():
        for nama_agent in daftar_agent:
            if nama_agent.lower() == nama_data_baru_sementara.lower():
                print(f"Agent sudah terdaftar pada role {nama_role}!")
                jeda()
                return
    agents[role][nama_data_baru_sementara] = True
    print(f"Agent '{nama_data_baru_sementara}' berhasil ditambahkan ke
{role}.")
    jeda()

def rekap_pengguna():
    clear()
    judul(" REKAP SEMUA PENGGUNA ")
    if len(data_performa) == 0:
        print("Belum ada data performa pengguna.")
    else:
        for username, data in data_performa.items():
            print(f"Username: {username} | Total Game:
{hitung_total_game(list(data['history_catat'].keys()))}")
    jeda()
```

H. Penerapan Error Handling

Semua input dilindungi try/except/else. Input yang salah memunculkan ValueError (atau KeyError pada pemilihan nomor game), pesan kesalahan ditampilkan, dan input diminta ulang lewat while True. Nilai hanya dikembalikan lewat blok else setelah semua pemeriksaan lolos.

```
def input_angka(pesan, minimum=0, maksimum=None):
    while True:
        try:
```

```
            angka = int(input(pesan).strip())
            if angka < minimum:
                raise ValueError(f"Angka minimal {minimum}.")
            if maksimum is not None and angka > maksimum:
                raise ValueError(f"Angka maksimal {maksimum}.")
        except ValueError as error:
            print(f"Input tidak valid: {error}")
        else:
            return angka
def pilih_nomor_game(history, aksi):
    while True:
        teks = input(f"\nNomor game untuk {aksi} (atau 'kembali'):
").strip().lower()

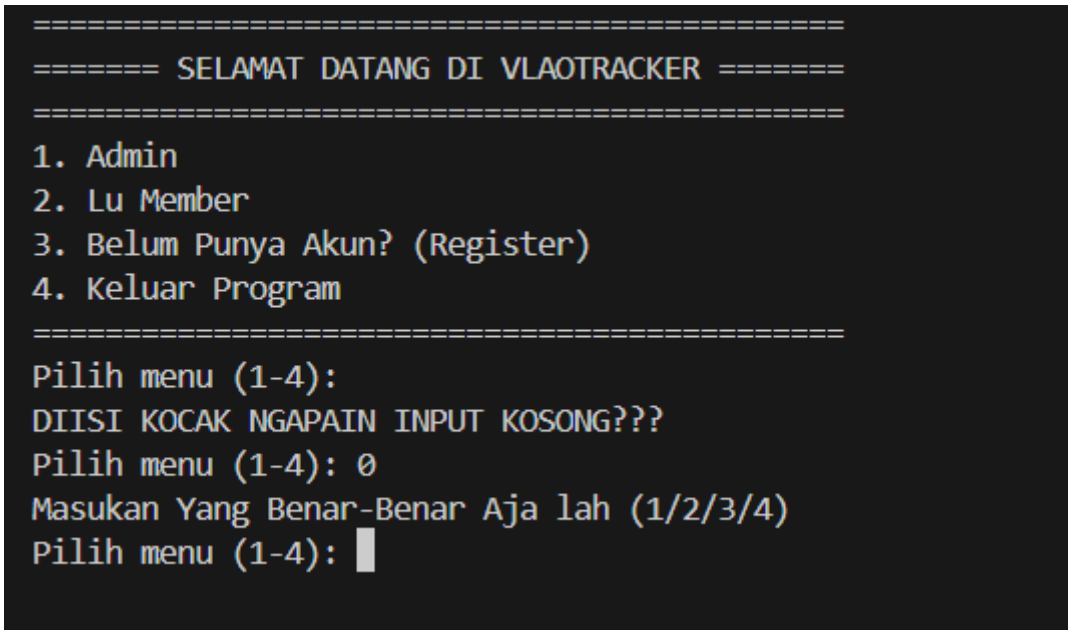
        if teks == "kembali":
            return None
        try:
            nomor = int(teks)
            if nomor not in history:
                raise KeyError(f"Game #{nomor} tidak ditemukan!")
        except ValueError:
            print("ValueError: masukkan nomor berupa angka atau
'kembali'.")
        except KeyError as error:
            print(f"KeyError: {error}")
        else:
            return nomor
```

I. Penerapan Fungsi Rekursif

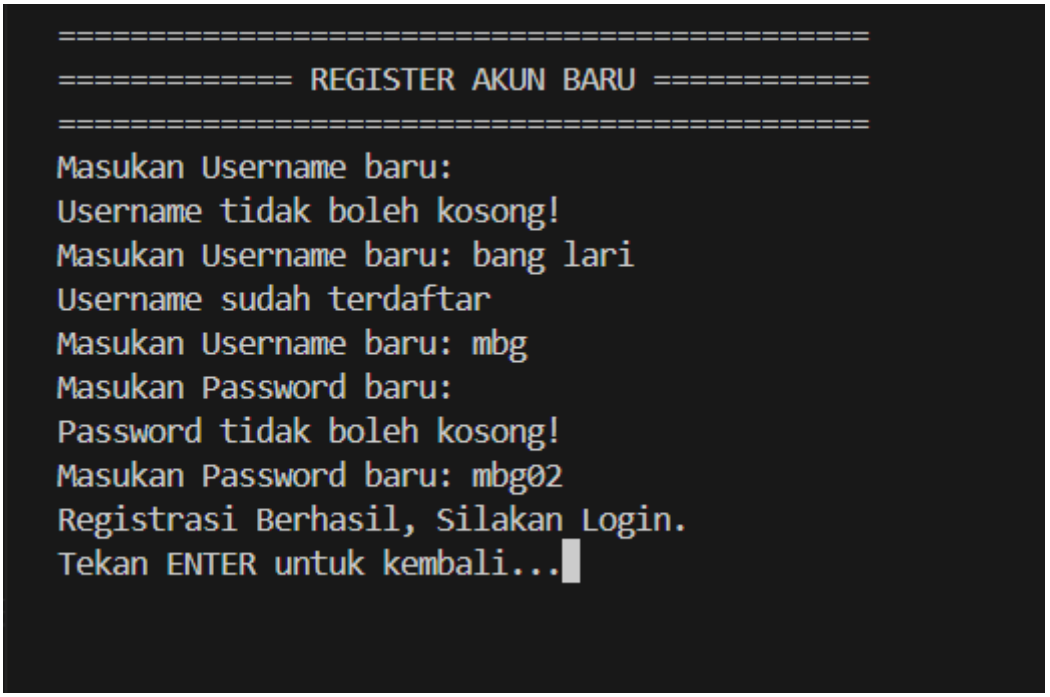
Fungsi ini menghitung jumlah game dengan memanggil dirinya sendiri. Base case-nya adalah saat indeks sudah mencapai panjang daftar, yang mengembalikan 0. Selama belum tercapai, fungsi mengembalikan 1 ditambah hasil pemanggilan dengan indeks berikutnya. Fungsi ini dipakai di statistik() dan rekap_pengguna().

```
def hitung_total_game(daftar_nomor, indeks=0):
    if indeks >= len(daftar_nomor):
        return 0
    else:
        return 1 + hitung_total_game(daftar_nomor, indeks + 1)
```

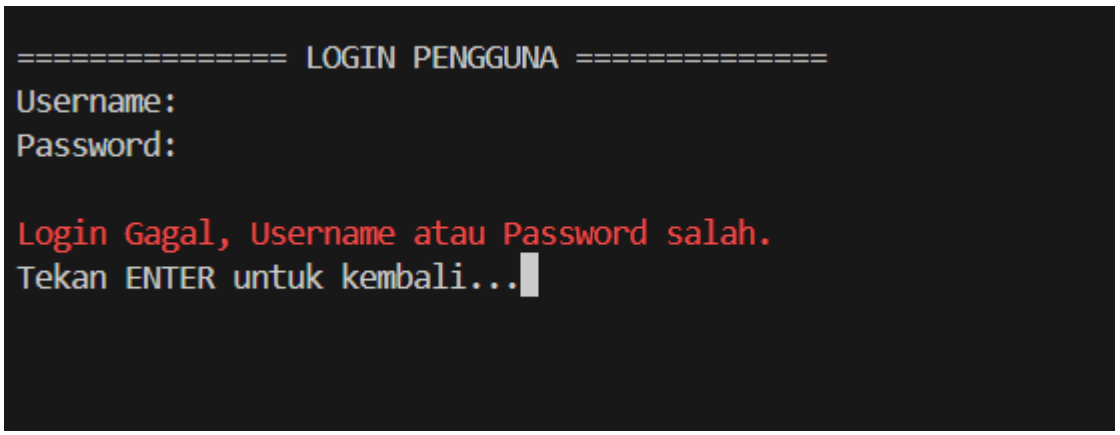
4. Hasil Output



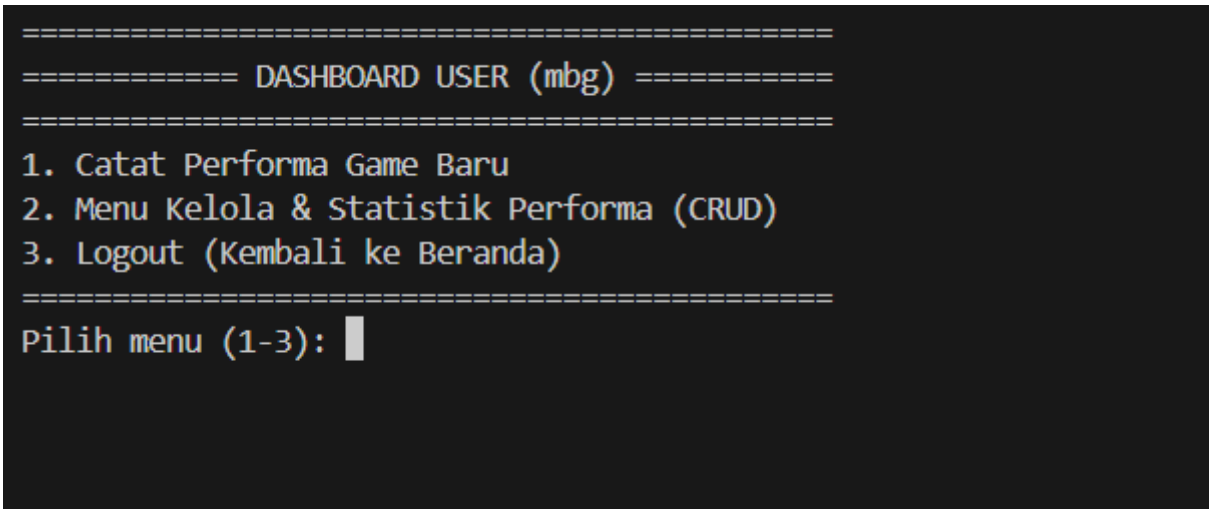
Gambar 4.1 Screenshot Output Terminal Menu Awal dan Validasi Input



Gambar 4.2 Screenshot Validasi Register dan Registrasi Berhasil



Gambar 4.3 Screenshot Output Terminal Login Gagal



Gambar 4.4 Screenshot Output Terminal Login Berhasil

```
=== PENCATATAN PERFORMA VALORANT DI VLAOTRACKER ===

Masukkan jumlah game yang ingin dicatat: 2

Pilih Rank Saat Ini:
1. Iron
2. Bronze
3. Silver
4. Gold
5. Platinum
6. Diamond
7. Ascendant
8. Immortal
9. Radiant
Pilih nomor rank (1-9): 0
Pilihan tidak valid!

Pilih Rank Saat Ini:
1. Iron
2. Bronze
3. Silver
4. Gold
5. Platinum
6. Diamond
7. Ascendant
8. Immortal
9. Radiant
Pilih nomor rank (1-9): 1
Masukkan RR saat ini (0-99): 150
RR maksimal 99! RR 100 ke atas hanya untuk rank Radiant.
Masukkan RR saat ini (0-99): 90
```

Gambar 4.5 Screenshot Output Terminal Input Jumlah Game, Rank, validasi Rank, validasi RR dan RR Awal



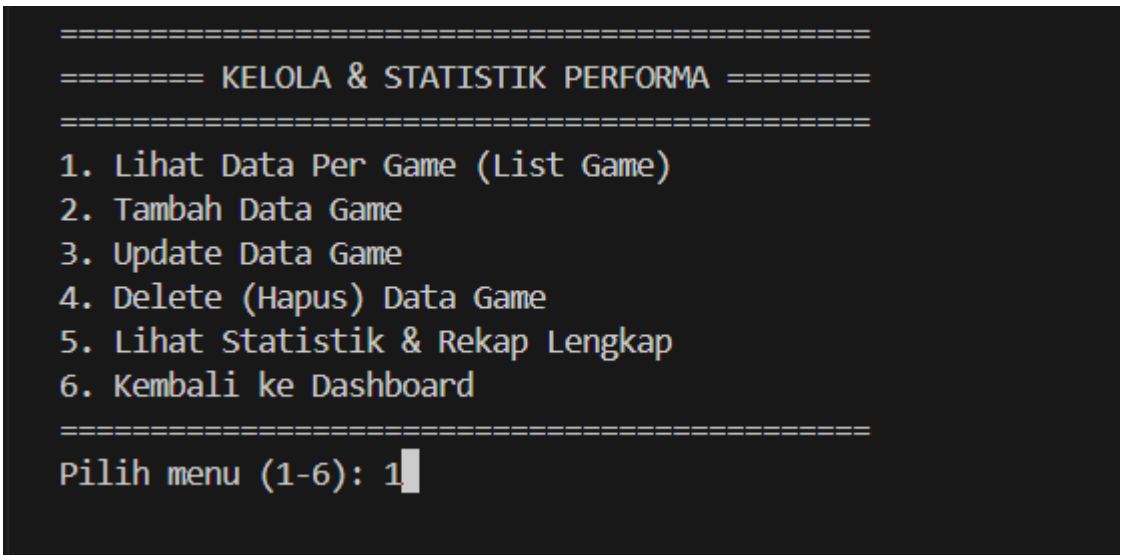
Gambar 4.6 Screenshot Output Terminal Validasi Jumlah Round, Validasi KDA, Validasi RR Yang Didapat.



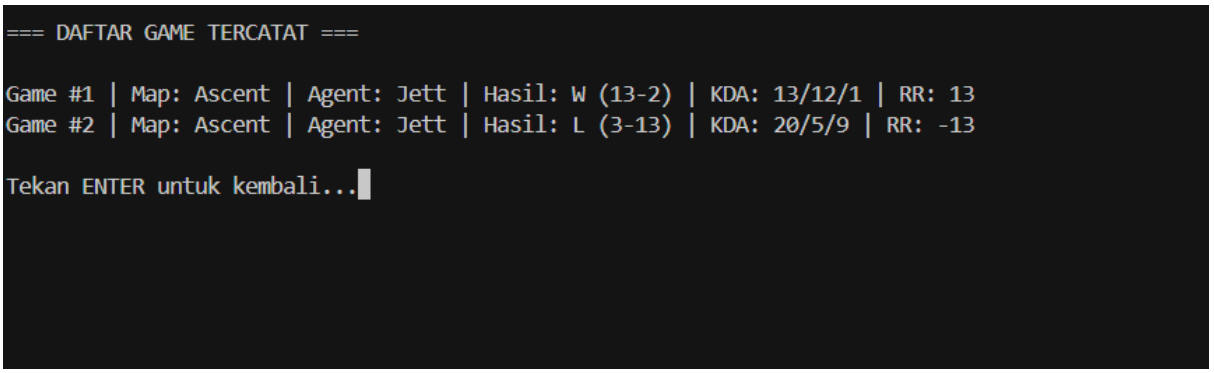
Gambar 4.7 Screenshot Output Terminal Validasi Map, Validasi Role



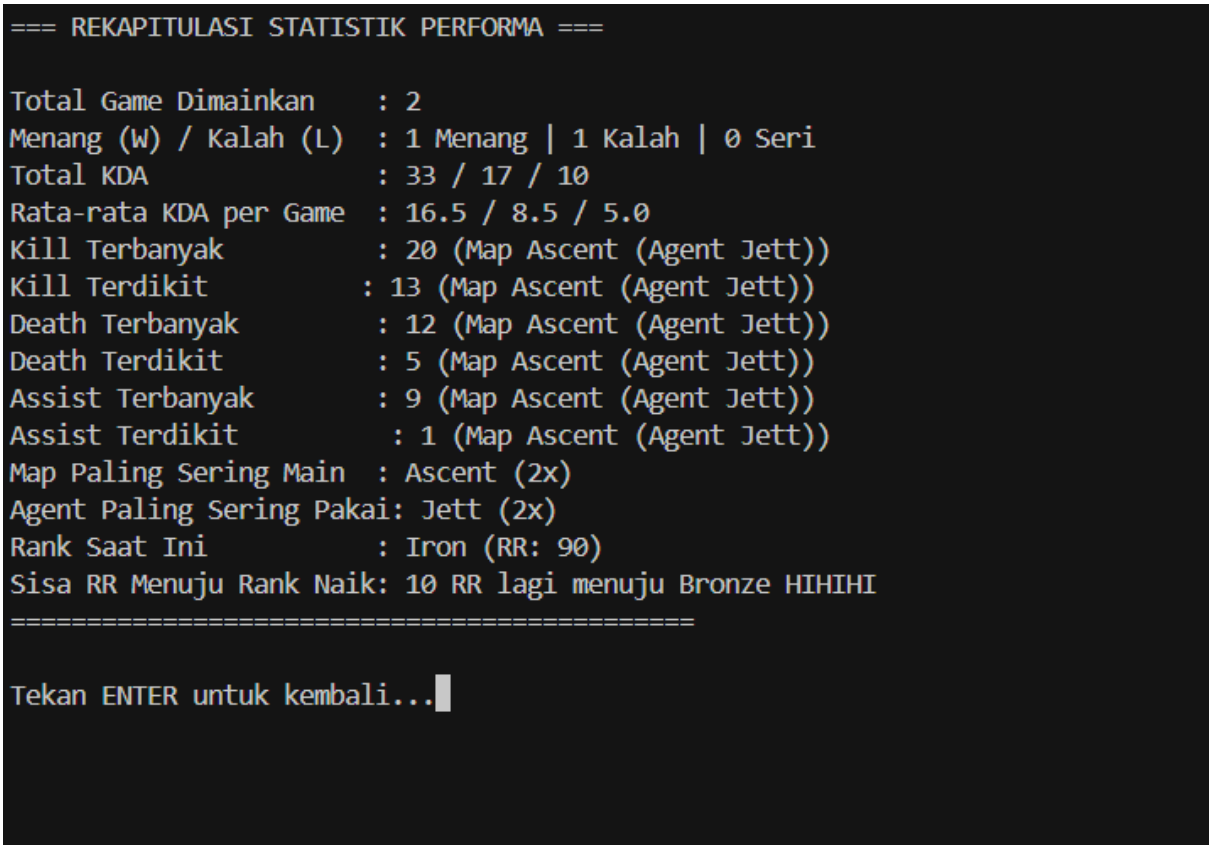
Gambar 4.8 Screenshot Output Terminal Validasi Agent dan Pencatatan Game Berhasil



Gambar 4.8 Screenshot Output Terminal Form CRUD



Gambar 4.9 Screenshot Output Terminal Read Awal



Gambar 4.10 Screenshot Output Terminal Rekap Awal



Gambar 4.11 Screenshot Output Terminal Create Data Baru

```
==========================================
MENU TAMBAH DATA GAME
==========================================

Masukkan jumlah game yang ingin dicatat: 1

--- GAME KE-1 DARI 1 ---

Daftar Map:
1. Ascent
2. Bind
3. Haven
4. Split
5. Lotus
6. Sunset
7. Abyss
8. Corrode
9. Icebox
10. Summit
11. Pearl
12. Fracture
13. Breeze
Pilih nomor map: 2

Pilih Role Agent:
1. Duelist
2. Initiator
3. Controller
4. Sentinel
Pilih nomor role (1-4): 1
```

Gambar 4.12 Screenshot Output Terminal Form Create Data Baru

```
=== DAFTAR GAME TERCATAT ===

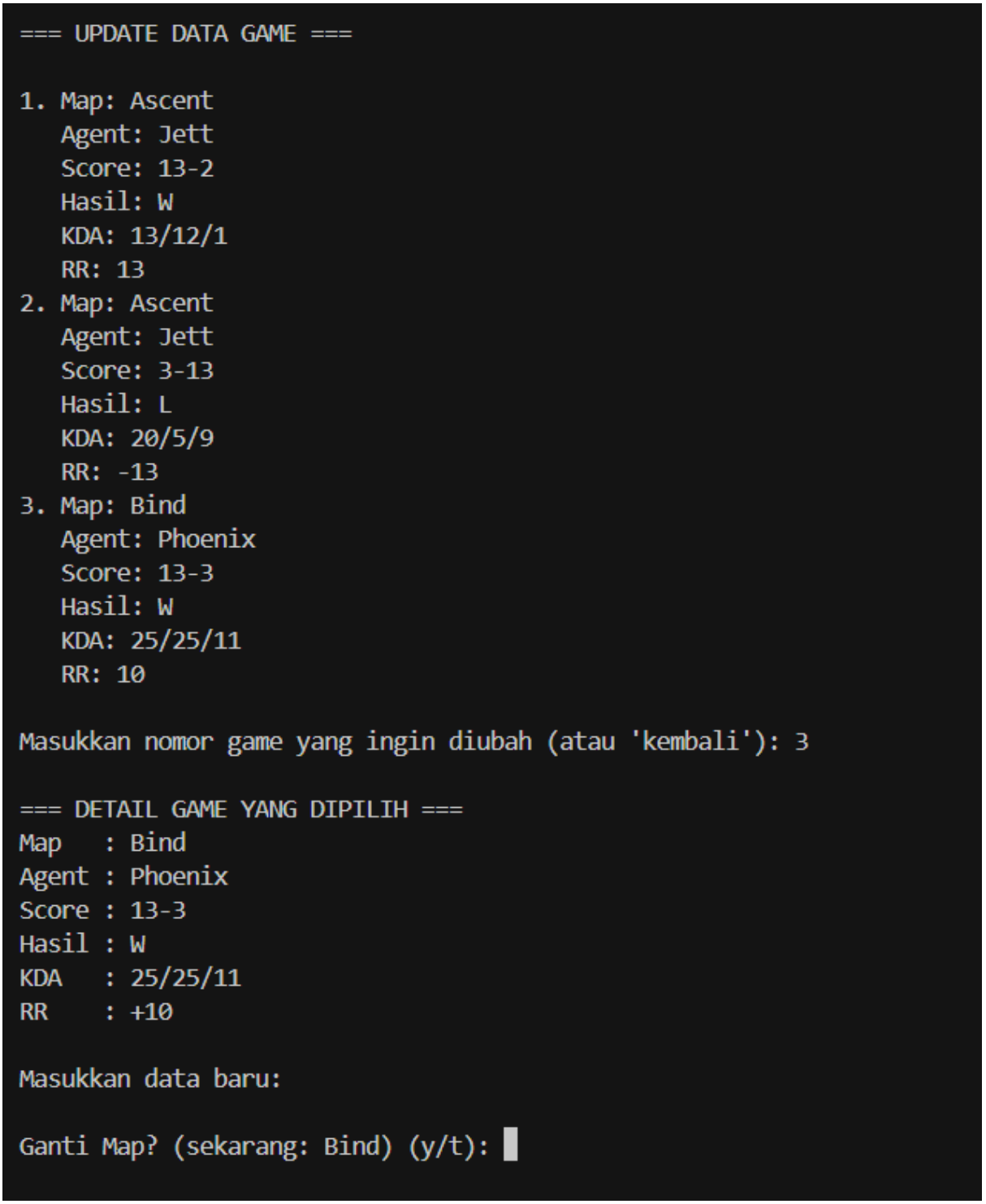
Game #1 | Map: Ascent | Agent: Jett | Hasil: W (13-2) | KDA: 13/12/1 | RR: 13
Game #2 | Map: Ascent | Agent: Jett | Hasil: L (3-13) | KDA: 20/5/9 | RR: -13
Game #3 | Map: Bind | Agent: Phoenix | Hasil: W (13-3) | KDA: 25/25/11 | RR: 10

Tekan ENTER untuk kembali...
```

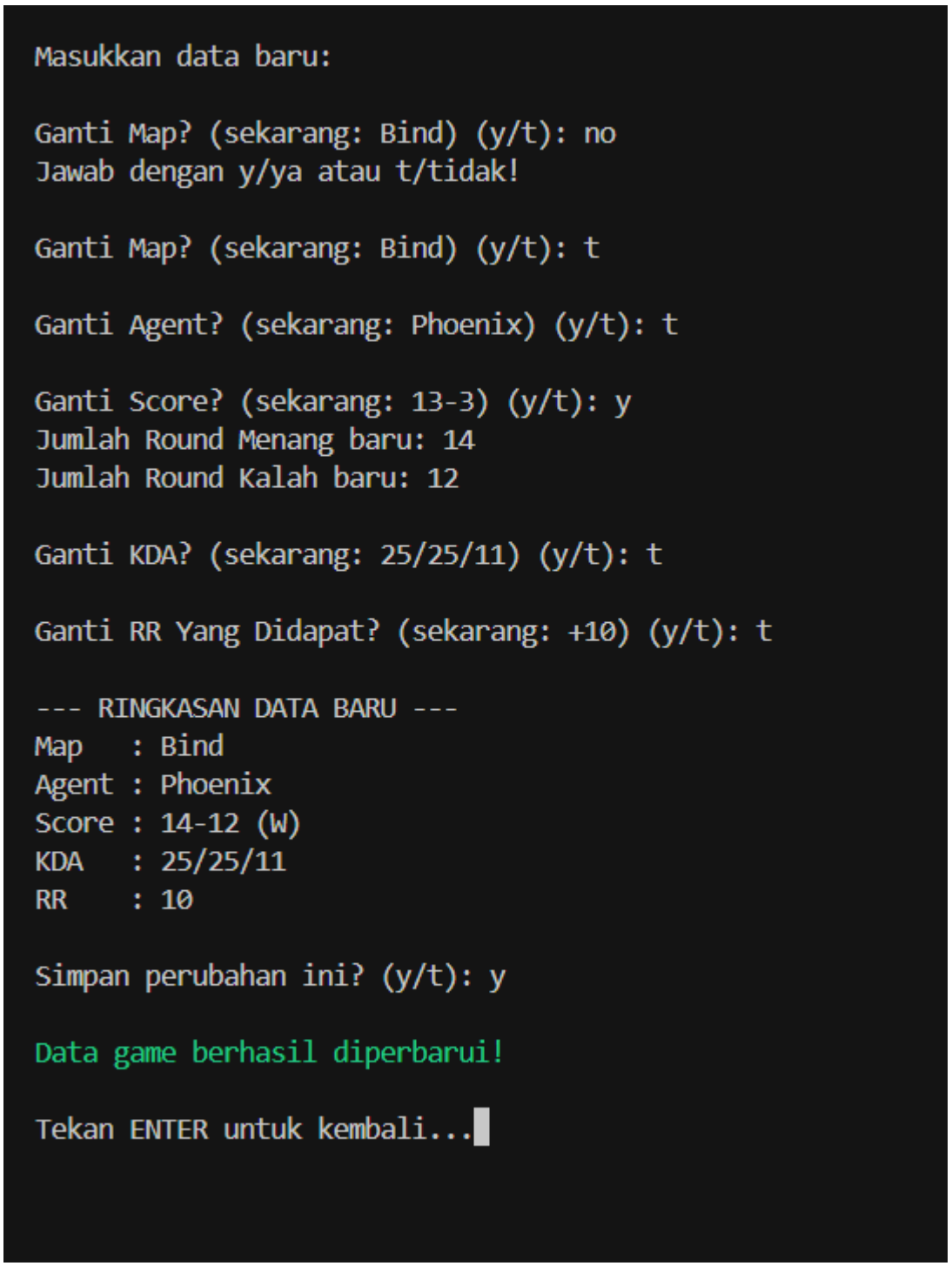
Gambar 4.13 Screenshot Output Terminal Read Setelah Create



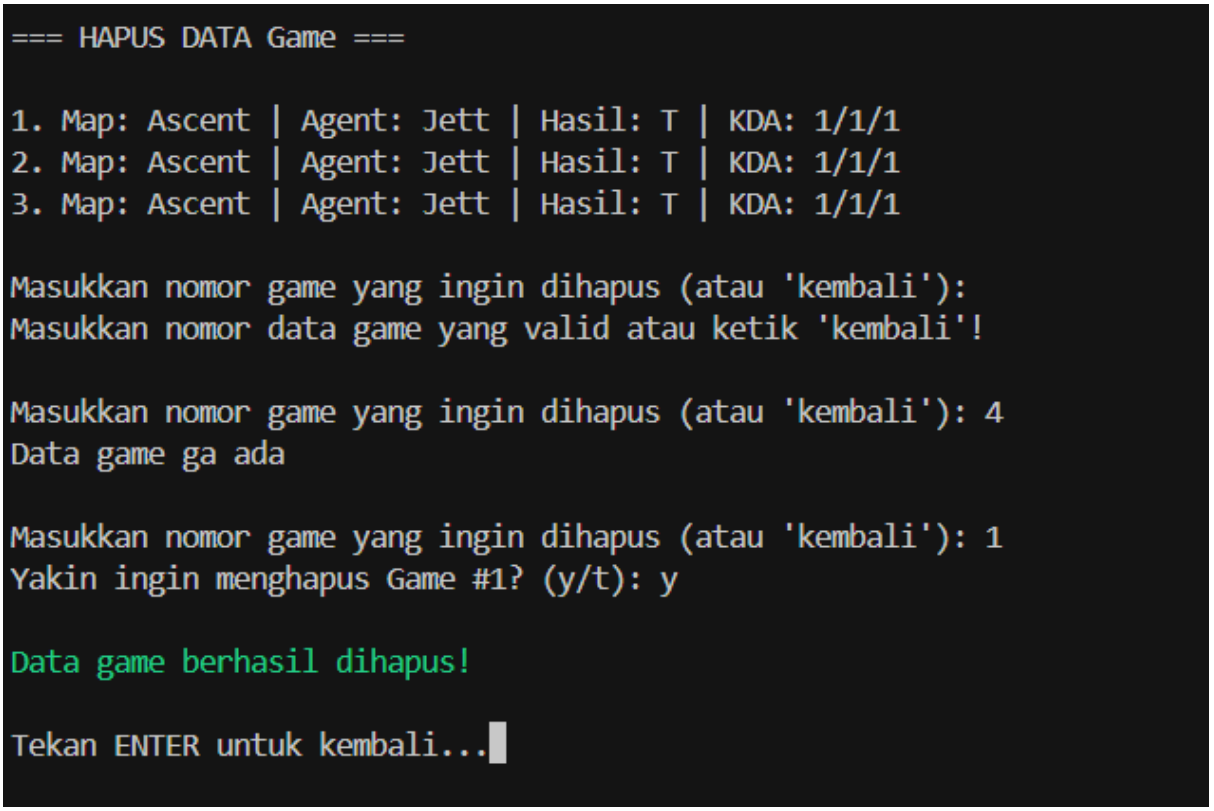
Gambar 4.14 Screenshot Output Terminal Rekap Setelah Create



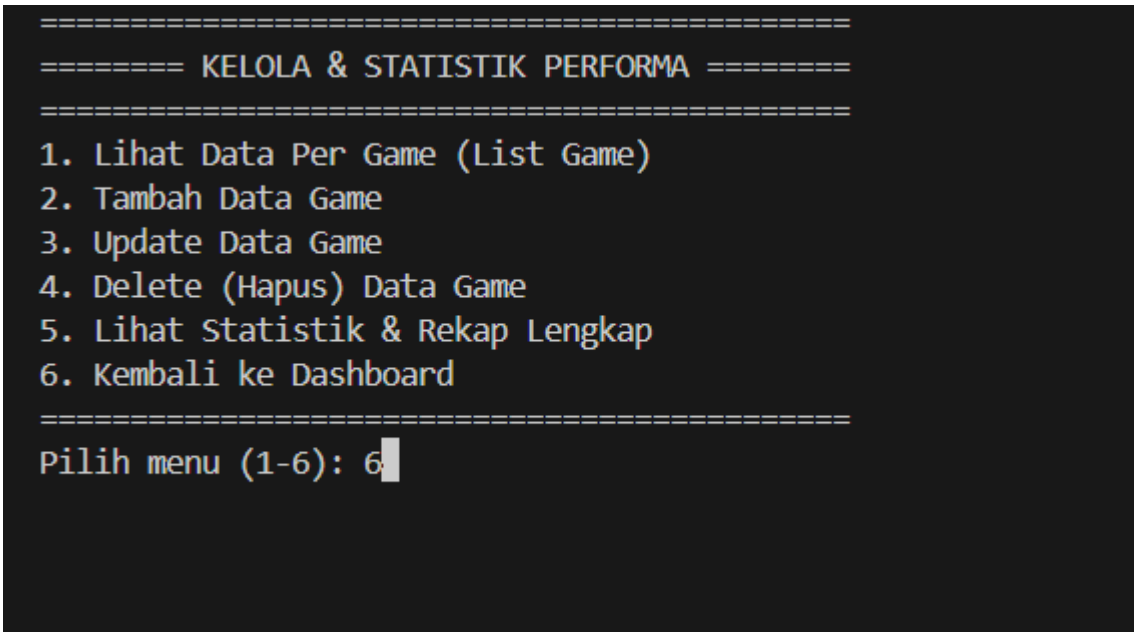
Gambar 4.15 Screenshot Output Terminal Update Data 3



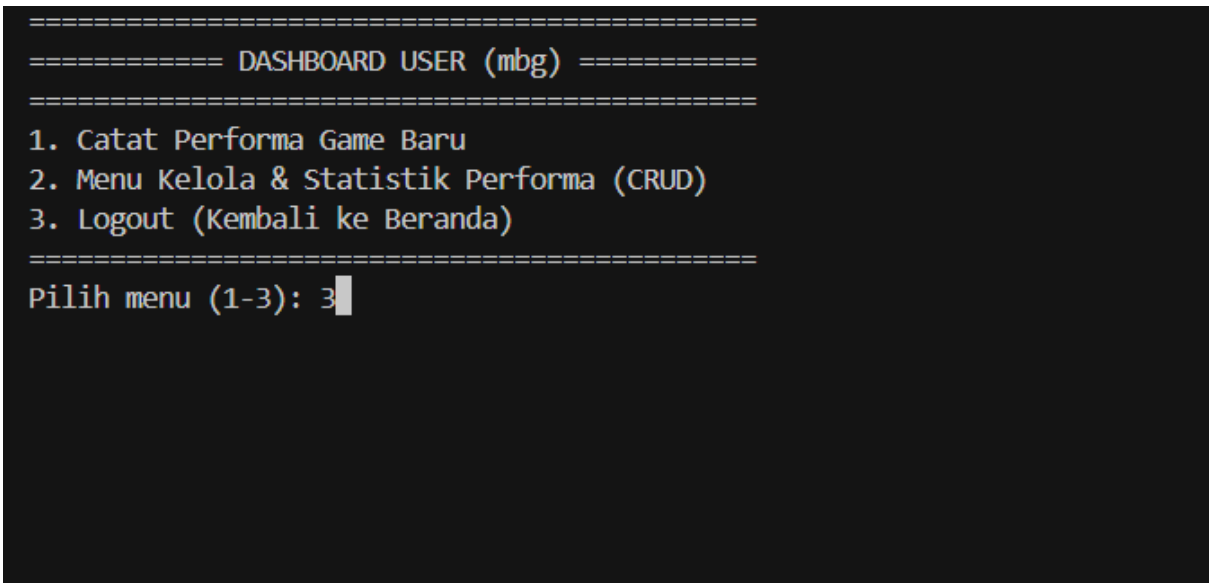
Gambar 4.16 Screenshot Output Terminal Form Update dan Berhasil



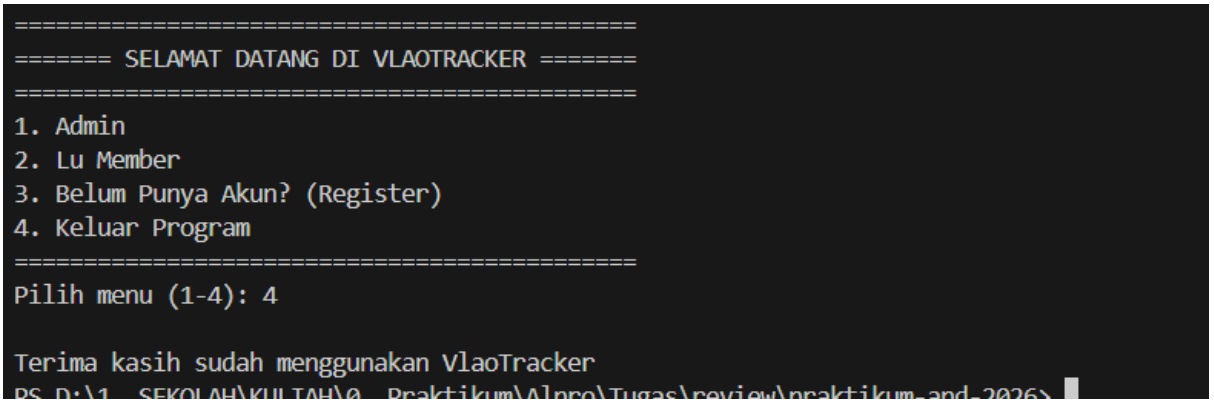
Gambar 4.17 Screenshot Output Terminal Form Delete, Validasi Delete dan Delete Berhasil



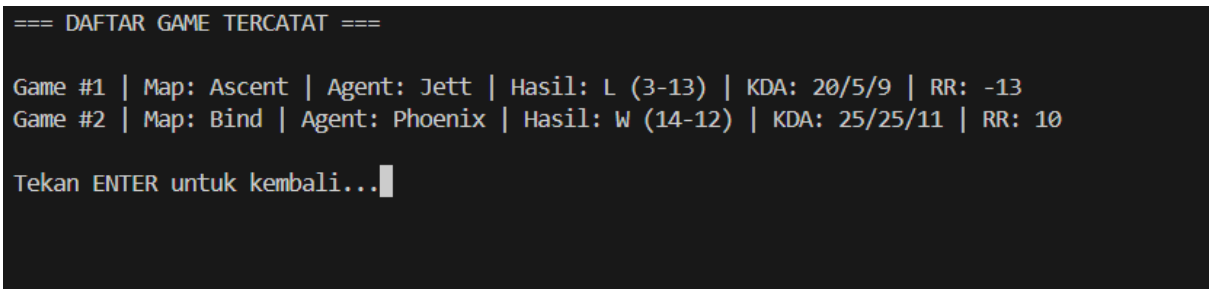
Gambar 4.18 Screenshot Output Terminal Konfirmasi T dan Rekap



Gambar 4.19 Screenshot Output Terminal Konfirmasi T dan Rekap

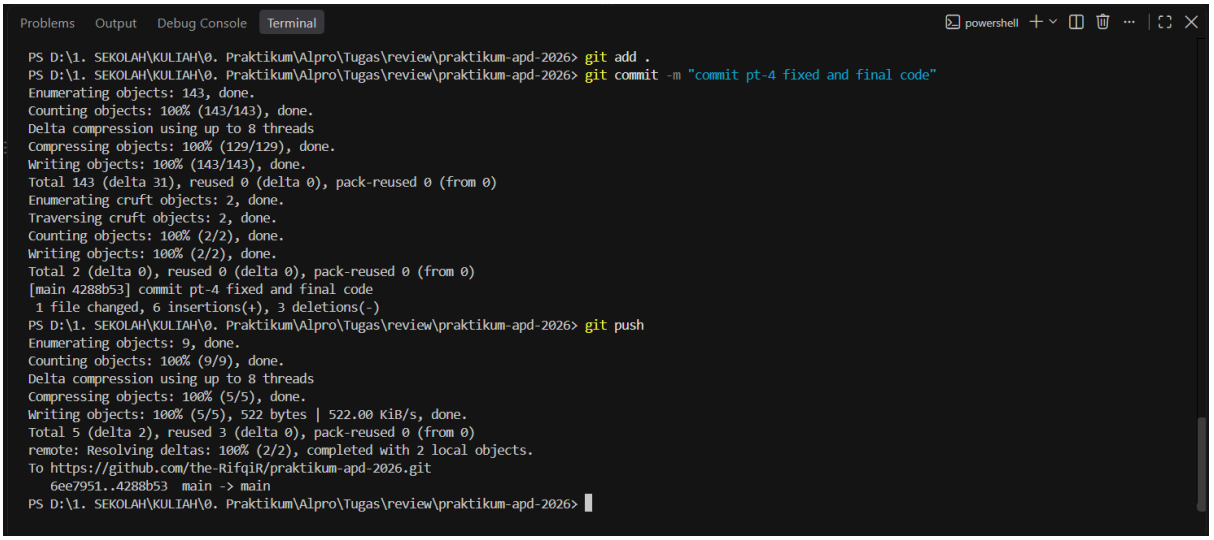


Gambar 4.20 Screenshot Output Terminal Konfirmasi T dan Rekap



Gambar 4.20 Screenshot Output Terminal Read Setelah Delete

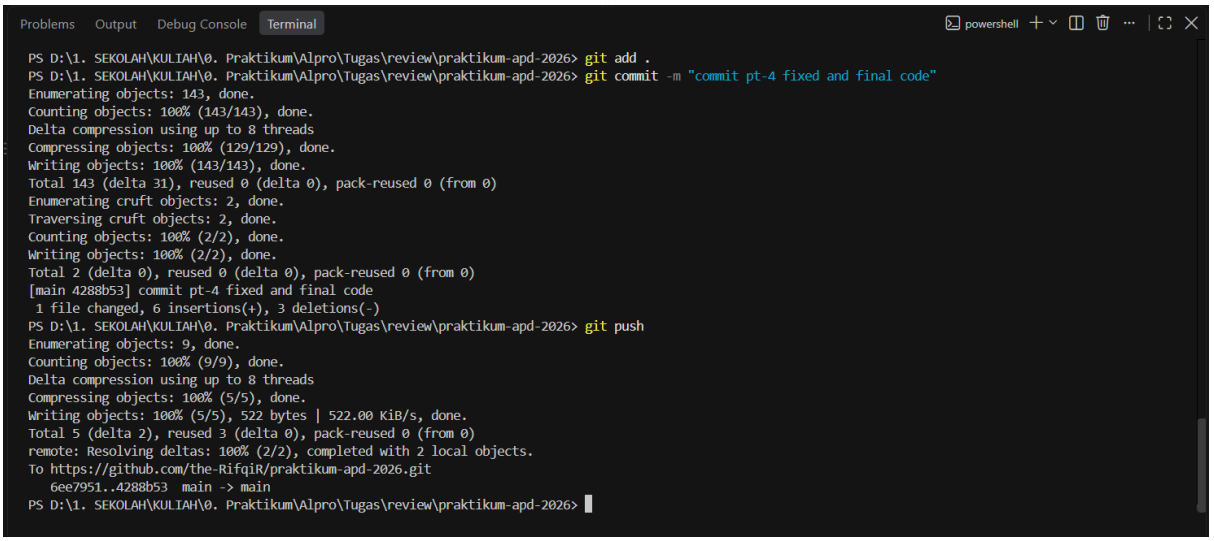
5. Git Add



Gambar 5.1 Screenshot Output Terminal Perintah git add

Penjelasannya sesuai nama “add” artinya menambahkan kalau git add . menambahkan semua perubahan ke dalam antrian git.

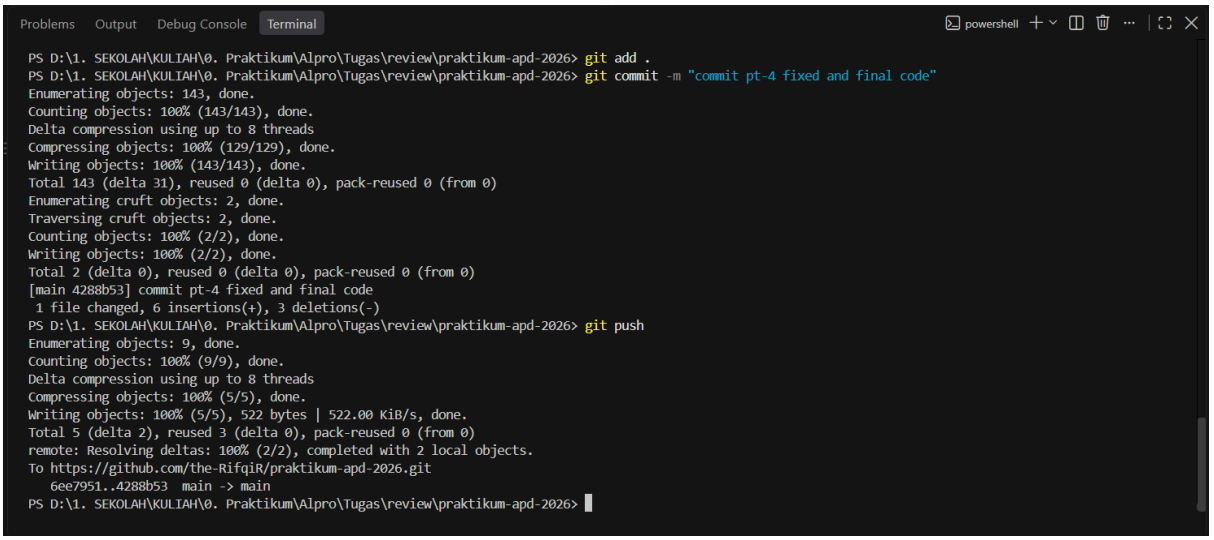
6. Git Commit



Gambar 5.2 Screenshot Output Terminal Perintah git commit

Commit di git buat nandain perubahan yang akan dikirim kegithub dengan memberi inisial commitnya.

7. Git Push



Gambar 5.3 Screenshot Output Terminal Perintah git push

Git push, mengirimkan data yang sudah ditambahkan dan diinisialisasikan ke github.